

JOBSHEET IV

Mata Kuliah: Praktikum Pemrograman Berbasis Objek

Relasi Kelas dan Array Objek

Dosen Pengampu:

Dian Hanifudin Subhi, S.Kom., M.Kom.



Disusun oleh:

Ahmad Rafid Riqkullah

254107020078

T1 – 2B

D-IV Teknik Informatika

Tanggal: 18 September 2026

POLITEKNIK NEGERI MALANG

Jl. Soekarno Hatta No.9, Jatimulyo, Kec. Lowokwaru, Kota Malang, Jawa Timur

65141

TAHUN 2026-2027

C. Langkah Kerja

Langkah 1: Account Berelasi dengan Customer

Code:

```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Costumer {
4     private String name;
5     private String address;
6
7     public Costumer(String name, String address) {
8         this.name = name;
9         this.address = address;
10    }
11
12    public String getName() {
13        return name;
14    }
15
16    public String getAddress() {
17        return address;
18    }
19 }
20
```

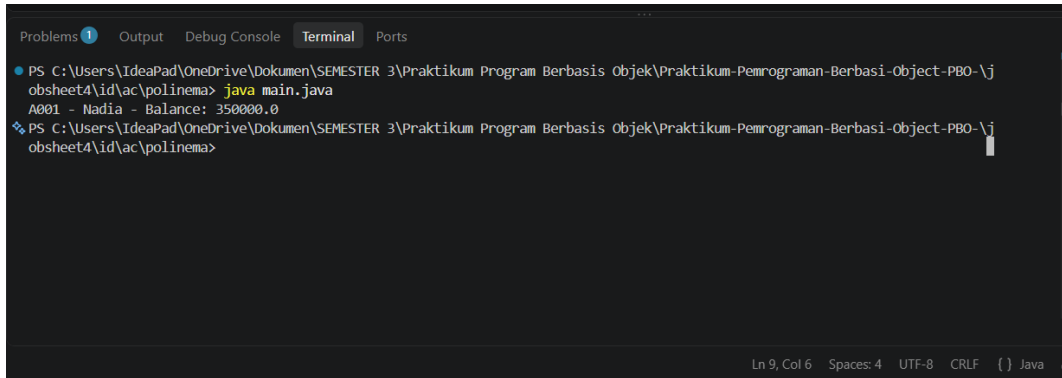
```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Account {
4     private String accountNumber;
5     private Costumer owner;
6     private double balance;
7     // private double dailyWithdrawallimit;
8
9     public Account(String accountNumber, Costumer owner, double balance) {
10        this.accountNumber = accountNumber;
11        this.balance = balance;
12        this.owner = owner;
13    }
14
15    public String getAccountNumber() {
16        return accountNumber;
17    }
18
19    public Costumer getOwner() {
20        return owner;
21    }
22    // public String getOwnerName() {
23    //     return ownerName;
24    // }
25    public double getBalance() {
26        return balance;
27    }
28
29    public boolean deposit(double amount) {
30        if (amount <= 0) {
31            return false;
32        }
33        balance += amount;
34        return true;
35    }
36
37    public boolean withdraw(double amount) {
38        if (amount <= 0 || amount > balance) {
39            return false;
40        }
41        balance -= amount;
42        return true;
43    }
44
45    public void printInfo() {
46        System.out.println(accountNumber + " - " + owner.getName() + " - Balance: " + balance);
47    }
48 }
49
50
```

```

1  package Jobsheet4.id.ac.polinema;
2
3  public class Main {
4      public static void main(String[] args) {
5          Costumer costumer1 = new Costumer("Nadia", "0812-0000-0001");
6          Account acc1 = new Account("A001", costumer1, 500000);
7          acc1.withdraw(150000);
8          acc1.printInfo();
9      }
10 }
11

```

Hasil:



```

Problems 1 Output Debug Console Terminal Ports
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema> java main.java
A001 - Nadia - Balance: 350000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema>
Ln 9, Col 6 Spaces: 4 UTF-8 CRLF {} Java

```

Penjelasan: Di langkah ini, kita mengubah cara kelas Account menyimpan data pemiliknya. Sebelumnya pemilik hanya disimpan sebagai teks biasa (String), tapi sekarang diganti menjadi sebuah objek dari kelas Costumer (ini disebut relasi *association*). Dengan begini, data pemilik jadi lebih lengkap karena dari dalam Account, kita tidak hanya tahu namanya, tapi juga bisa memanggil data alamat dari objek Costumer tersebut.

Langkah 2: Bank Mengelola Banyak Account

Code:

```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Bank {
4     private Account[] accounts;
5     private int count;
6
7     public Bank(int capacity) {
8         accounts = new Account[capacity];
9         count = 0;
10    }
11
12    public boolean addAccount(Account account) {
13        if (count >= accounts.length) {
14            return false;
15        }
16        accounts[count] = account;
17        count++;
18        return true;
19    }
20
21    public Account findAccount(String accountNumber) {
22        for (int i = 0; i < count; i++) {
23            if (accounts[i].getAccountNumber().equals(accountNumber)) {
24                return accounts[i];
25            }
26        }
27        return null;
28    }
29
30    public void printAllAccounts() {
31        for (int i = 0; i < count; i++) {
32            accounts[i].printInfo();
33        }
34    }
35 }
36
```

```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Costumer costumer1 = new Costumer("Nadia", "0812-0000-0001");
6         Account acc1 = new Account("A001", costumer1, 500000);
7         acc1.withdraw(150000);
8
9         Costumer costumer2 = new Costumer("Sari", "0812-0000-0002");
10        Account acc2 = new Account("A002", costumer2, 200000);
11
12        Bank bank = new Bank(10);
13        bank.addAccount(acc1);
14        bank.addAccount(acc2);
15        bank.printAllAccounts();
16
17        Account found = bank.findAccount("A002");
18        if(found != null) {
19            found.printInfo();
20        }
21    }
22 }
23
```

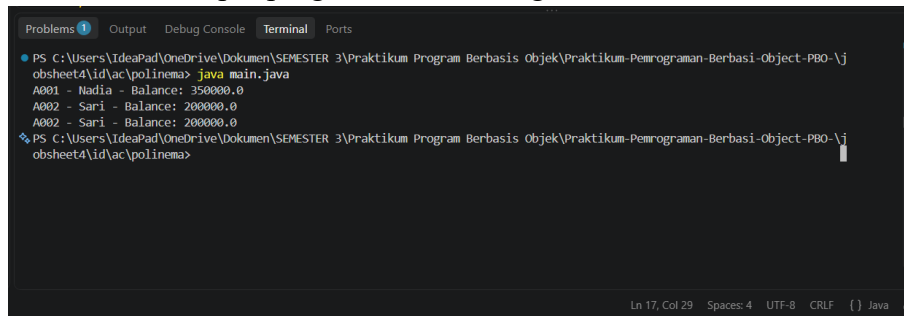
Hasil:

```
Problems 1 Output Debug Console Terminal Ports
• PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema> java main.java
A001 - Nadia - Balance: 350000.0
A002 - Sari - Balance: 200000.0
A002 - Sari - Balance: 200000.0
❖ PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema>
```

Penjelasan: Pada bagian ini, kita membuat kelas baru bernama Bank yang tugasnya menjadi wadah pengelola banyak rekening (Account). Bank menyimpan daftar rekening tersebut di dalam sebuah *array* (ini disebut relasi *aggregation*). Di dalamnya, kita juga membuatkan fungsi tambahan agar Bank bisa memasukkan rekening baru, serta mencari rekening secara spesifik berdasarkan nomor rekeningnya.

D. Tugas dan Hasil Kerja

- Screenshot output program setelah Langkah 2.



```
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum Pemrograman Berbasis Object-PBO-Vj\obsheet4\id\ac\polinema> java main.java
A001 - Nadia - Balance: 350000.0
A002 - Sari - Balance: 200000.0
A002 - Sari - Balance: 200000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum Pemrograman Berbasis Object-PBO-Vj\obsheet4\id\ac\polinema>
```

- Tugas mandiri:
1. Seorang customer bisa saja memiliki lebih dari satu rekening. Tambahkan method `findAccountsByOwnerName(String name)` ke `Bank`, yang mengembalikan array berisi seluruh `Account` milik customer dengan nama tersebut:



```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Bank {
4     private Account[] accounts;
5     private int count;
6
7     public Bank(int capacity) {
8         accounts = new Account[capacity];
9         count = 0;
10    }
11
12    public boolean addAccount(Account account) {
13        if (count >= accounts.length) {
14            return false;
15        }
16        accounts[count] = account;
17        count++;
18        return true;
19    }
20
21    public Account findAccount(String accountNumber) {
22        for (int i = 0; i < count; i++) {
23            if (accounts[i].getAccountNumber().equals(accountNumber)) {
24                return accounts[i];
25            }
26        }
27        return null;
28    }
29
30    public Account[] findAccountsByOwner(String Name) {
31        int matchCount = 0;
32        for (int i = 0; i < count; i++) {
33            if (accounts[i].getOwner().getName().equals(Name)) {
34                matchCount++;
35            }
36        }
37        Account[] result = new Account[matchCount];
38        int index = 0;
39        for (int i = 0; i < count; i++) {
40            if (accounts[i].getOwner().getName().equals(Name)) {
41                result[index] = accounts[i];
42                index++;
43            }
44        }
45        return result;
46    }
47
48    public void printAllAccounts() {
49        for (int i = 0; i < count; i++) {
50            accounts[i].printInfo();
51        }
52    }
53 }
54
```

```
1 package Jobsheet4.id.ac.polinema;
2
3 public class Main {
4     public static void main(String[] args) {
5         Costumer nadia = new Costumer("Nadia", "0812-0000-0001");
6         Account acc1 = new Account("A001", nadia, 500000);
7         Account acc2 = new Account("A002", nadia, 300000);
8
9         Costumer sari = new Costumer("Sari", "0812-0000-0002");
10        Account acc3 = new Account("A003", sari, 200000);
11
12        Bank bank = new Bank(10);
13        bank.addAccount(acc1);
14        bank.addAccount(acc2);
15        bank.addAccount(acc3);
16
17        Account[] nadiaAccounts = bank.findAccountsByOwner("Nadia");
18
19        System.out.println("Jumlah akun: " + nadiaAccounts.length);
20        for (int i = 0; i < nadiaAccounts.length; i++) {
21            nadiaAccounts[i].printInfo();
22        }
23    }
24 }
```

Buktikan dengan membuat satu customer bernama “Nadia” yang memiliki dua Account, satu customer lain dengan satu Account, memasukkan ketiganya ke Bank, lalu memanggil `findAccountsByOwnerName("Nadia")` dan mencetak jumlah serta isi hasilnya:

```
Problems 1 Output Debug Console Terminal Ports
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema> java main.java
Jumlah akun: 2
A001 - Nadia - Balance: 500000.0
A002 - Nadia - Balance: 300000.0
PS C:\Users\IdeaPad\OneDrive\Dokumen\SEMESTER 3\Praktikum Program Berbasis Objek\Praktikum-Pemrograman-Berbasi-Object-PBO-\j
obsheet4\id\ac\polinema>
```

Penjelasan: Kode ini berfungsi untuk mencari dan menampilkan semua rekening yang dimiliki oleh satu orang yang sama. Cara kerjanya dibagi dua tahap: pertama, program menghitung dulu ada berapa jumlah rekening yang namanya cocok (misal "Nadia"). Setelah jumlahnya ketemu, program akan membuat *array* baru sebesar jumlah tersebut, lalu memindahkan data rekening yang cocok tadi ke dalam *array* baru agar bisa ditampilkan semua secara bersamaan.

2. Jawab secara singkat (2-3 kalimat untuk masing-masing pertanyaan):
 - a. mengapa relasi Account-Customer disebut association, bukan aggregation atau composition?

Jawab: Relasi Account-Customer disebut *association* karena kedua kelas ini pada dasarnya mandiri dan bisa berdiri sendiri. Keduanya saling berhubungan, tapi tidak saling memiliki secara mutlak; misalnya,

jika sebuah akun bank ditutup, data personal pelanggannya tidak harus ikut dihapus.

- b. bayangkan objek Bank dihapus dari memori. Menurutmu, apakah objek Account yang pernah ditambahkan ke dalamnya seharusnya ikut terhapus, atau tetap bisa berdiri sendiri? Apa maksud jawabanmu terhadap jenis relasi Bank-Account?

Jawab: Jika objek Bank dihapus, objek Account seharusnya tidak ikut terhapus dan tetap bisa berdiri sendiri. Inilah makna dari relasi *aggregation* antara Bank dan Account, yaitu relasi "memiliki" (*has-a*) yang sifatnya lemah, di mana objek bagian (Account) bisa tetap eksis meskipun objek utamanya (Bank) sudah tidak ada.

Hasil dan Pembahasan

Pada praktikum ini, kode program telah berhasil diimplementasikan untuk memahami konsep relasi antar kelas dalam Pemrograman Berbasis Objek, khususnya *association* dan *aggregation*. Lewat relasi *association*, kelas Account sukses dihubungkan dengan kelas Costumer untuk saling melengkapi data. Sementara itu, penerapan relasi *aggregation* pada kelas Bank membuktikan bahwa sekumpulan objek Account bisa dikelola dengan rapi menggunakan *array of objects*. Seluruh kode berjalan tanpa *error*, dan fungsi pencarian data (baik mencari satu akun atau mencari beberapa akun dengan nama pemilik yang sama) berhasil mengeluarkan *output* sesuai dengan tujuan tugas.

Kesimpulan

Penggunaan relasi antar kelas seperti *association* dan *aggregation* sangat mempermudah kita dalam menerjemahkan hubungan objek di dunia nyata ke dalam kode program. Selain membuat kode lebih terstruktur, penggunaan *array of objects* juga terbukti sangat efektif sebagai media untuk menyimpan, mengelola, dan mencari sekumpulan data objek dalam jumlah banyak dengan lebih efisien.

Lampiran

[Link Github Ahmad Rafid Riqkullah](https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet4/id/ac/polinema) atau <https://github.com/Pidddd/Praktikum-Pemrograman-Berbasi-Object-PBO-/tree/main/Jobsheet4/id/ac/polinema>